Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 1: Azure SQL Managed Instance, one of Microsoft Azure's most powerful and flexible database services.

🧠 1. What is Azure SQL Managed Instance?

Azure SQL Managed Instance (MI) is a **Platform as a Service (PaaS)** offering under **Azure SQL Database** family.
It's designed to give you **the best of both worlds**:

- Almost **full SQL Server compatibility** (like on-premises SQL Server).
- **Cloud benefits** like high availability, automatic backups, scalability, and patching.

So you can **lift and shift** existing SQL Server databases to the cloud with minimal changes to your applications.

⚙️ 2. How It Fits in the Azure SQL Family

Service	Type	Best for
SQL Server on VM (IaaS)	Infrastructure-as-a-Service	Full control, manual management
Azure SQL Managed Instance (PaaS)	Platform-as-a-Service	Near full SQL Server compatibility + managed infrastructure
Azure SQL Database	Platform-as-a-Service	Modern, cloud-native apps with simpler schema

Managed Instance sits between **SQL Server on VM** and **Azure SQL Database** — giving you **control + management convenience**.

🏗️ 3. Architecture Overview

Each Managed Instance runs inside a **dedicated virtual network (VNet)**.
This provides **network isolation** and allows **private IP connectivity**, similar to an on-premises environment.

https://www.sqldbachamps.com

Main components:

1. **Virtual Cluster:** Underlying infrastructure that hosts your Managed Instances.
2. **Compute nodes:** Run your SQL databases (like SQL Engine).
3. **Storage layer:** Uses Azure Premium or Standard storage for data and logs.
4. **Control plane:** Handles management operations like scaling, patching, and backups.

💡 4. Key Features and Capabilities

🧩 SQL Server Compatibility

Supports nearly all SQL Server features:

- SQL Agent (scheduling jobs)
- Linked Servers
- CLR integration
- Service Broker
- Database Mail
- Cross-database queries
- Transparent Data Encryption (TDE)
- Native backup/restore using .bak files

This makes **migration from on-premises SQL Server** straightforward.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔒 Security

- **Private VNet access** — no public endpoint by default.
- **Azure Active Directory (AAD)** authentication.
- **Transparent Data Encryption (TDE)** enabled by default.
- **Always Encrypted**, auditing, and firewall rules.
- **Advanced Threat Protection** and **SQL Defender** integration.

☁️ High Availability & Disaster Recovery

- Built-in **automatic failover**.
- **99.99% availability SLA**.
- Supports **auto backups** (point-in-time restore up to 35 days).
- Supports **geo-replication** to another region for disaster recovery.

⚡ Performance & Scaling

You can choose between two service tiers:

Tier	Description	Use case
General Purpose (GP)	Standard Azure storage, compute + storage separated	Most business workloads
Business Critical (BC)	Premium SSD storage, compute + storage on same node for low latency	High-performance / mission-critical apps

Scaling:

- vCores: 4–128+
- Memory: Depends on vCore size
- Storage: Up to 8 TB per database

https://www.sqldbachamps.com

🔄 Automation

Managed Instance automatically handles:

- Patching (OS + SQL)
- Backups
- Failover
- Monitoring and metrics (via Azure Monitor)

You can also integrate it with:

- **Azure Automation** for custom runbooks
- **Azure Policy** for governance

🚀 5. Connectivity Options

You can connect to Managed Instance:

- From **on-premises** networks using VPN or ExpressRoute.
- From **Azure VMs** or **App Services** within the same VNet.
- From **public internet** (only if you enable public endpoint — not recommended for production).

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🔄 6. Migration to Managed Instance

Microsoft provides several tools:

- **Azure Database Migration Service (DMS)**
- **Transaction Log Shipping**
- **Backup/Restore using .bak files**
- **Data Migration Assistant (DMA)** — checks for compatibility issues.

Common migration path:

1. Assess with DMA.
2. Choose target tier (GP or BC).
3. Use DMS to migrate data with minimal downtime.

💰 7. Pricing Model

Azure SQL Managed Instance pricing is based on:

- **Compute (vCores)**
- **Storage**
- **Backup storage**
- **License model**:
  - **vCore-based model**
  - **Azure Hybrid Benefit** (bring your existing SQL Server licenses to save ~55%).

👉 Example:

A 16 vCore General Purpose instance costs roughly **$3–4K/month**, depending on region and licensing.

🧩 8. Use Cases

https://www.sqldbachamps.com

- **Lift-and-shift migrations** from SQL Server 2012/2016/2019.
- **Modernizing legacy applications** with minimal code changes.
- **Secure, high-performance enterprise databases**.
- **Hybrid cloud setups** with VPN/ExpressRoute connectivity.

🛠️ 9. Tools for Management

You can manage Azure SQL Managed Instance using:

- **Azure Portal**
- **Azure CLI**
- **PowerShell**
- **T-SQL**
- **SQL Server Management Studio (SSMS)**
- **Azure Data Studio**

📜 10. Example: Basic Deployment Steps

Here's a quick view of how to deploy a Managed Instance:

1. Go to **Azure Portal → Create resource → Databases → Azure SQL Managed Instance**.
2. Choose:
   - Subscription & Resource Group
   - Name, region, compute tier (GP/BC)

- vCores & storage size
- VNet & subnet (must be preconfigured)

3. Configure:
   - Admin login
   - Backup retention
   - Maintenance window
4. Review + Create.
5. Connect via SSMS using the **private IP** or **DNS name**.

🔍 11. Pros & Cons Summary

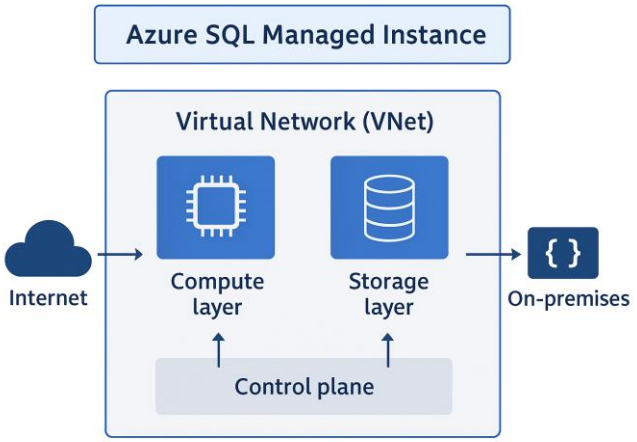
Pros	Cons
Full SQL Server compatibility	Longer provisioning time (1–2 hrs)
Fully managed (patching, backup)	Must use VNet (more setup)
High availability built-in	Higher cost than Azure SQL DB
Scalable & secure	Limited access to OS-level customization

🎓 Summary

Azure SQL Managed Instance = Full SQL Server in the cloud, managed by Microsoft.

✅ Ideal for enterprises migrating legacy SQL workloads.
✅ Offers performance, compatibility, and automation.
✅ Combines on-premises control with cloud simplicity.

https://www.sqldbachamps.com



Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 2: Managed Instance Deep Dive (Hands-On).

☁️ What Is Azure SQL Managed Instance (MI)?

Azure SQL Managed Instance =

👉 "Full SQL Server instance managed by Microsoft" (no OS or patching hassles)

👉 but **you get full SQL feature compatibility** — SQL Agent, Linked Servers, CLR, DB Mail, etc.

It sits **between**:

- Azure SQL Database (light, modern, limited)
- SQL Server on Azure VM (full control, heavy admin)

🧱 1. Architecture & Core Concepts

Component	Description
Compute Node	Your MI (dedicated cores, memory, tempdb, system DBs)
Storage	Azure premium remote storage with auto-grow
VNet Integration	Runs inside your private virtual network (VNet)
Service Tiers	General Purpose / Business Critical
HA	Built-in; always on secondary replicas under the hood

⚙️ Two Service Tiers

Tier	Storage	Use Case	Notes
General Purpose	Remote storage	Most workloads	Lower cost
Business Critical	Local SSD storage	High IOPS & low latency	Includes read-only replica

https://www.sqldbachamps.com

🧩 2. Key Differences: Azure SQL Database vs Managed Instance

Feature	Azure SQL Database	Managed Instance
SQL Agent	❌ No	✅ Yes
Linked Servers	❌ Limited	✅ Full support
Cross-Database Query	❌ No	✅ Yes
SSIS / CLR	❌ No	✅ Yes
DB Mail	❌ No	✅ Yes
Native Backup/Restore (.bak)	❌ No	✅ Yes
Network	Public endpoint	Private VNet
Target Use	SaaS or micro-app	Lift & Shift full instance

🧭 3. Hands-On: Create a Managed Instance (MI)

🧩 Step 1: Prerequisites

✅ Azure Subscription

✅ Virtual Network (VNet)

✅ Subnet dedicated for Managed Instance

✅ SSMS 18+ (latest version)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Step 2: Create Managed Instance (via Azure Portal)

1. Go to **Azure Portal → Create a resource → Databases → SQL Managed Instance**.
2. **Basics tab**
   - Subscription: Select your subscription
   - Resource Group: RG-AzureSQL-MI
   - Managed Instance name: sqlmi-demo2025
   - Region: *East US*
   - Compute + Storage:
     - Tier: **General Purpose**
     - vCores: 4
     - Storage: 64 GB
   - Authentication: SQL authentication (user: sqlmiadmin)
   - Enable **Public Endpoint (optional)** if you plan to connect via SSMS outside Azure.
3. **Networking tab**
   - Select VNet: VNET-MI
   - Subnet: SQLMI-Subnet (must be dedicated)
4. **Security tab**
   - Leave defaults (TDE enabled).
5. **Review + Create → Create.**

Deployment can take **4–6 hours** ⏳ (it's normal, MI builds internal infrastructure).

Step 3: Configure Public Endpoint (optional)

If you want to connect from SSMS on your laptop:

https://www.sqldbachamps.com

1. Go to your MI → **Networking → Public endpoint** → Enable.
2. Add your client IP under **Firewall rules**.

🖥️ **4. Connect via SSMS**

Once provisioned, note the **FQDN**:
sqlmi-demo2025.public.<region>.database.windows.net,3342

1. Open **SSMS**.
2. Server name → above FQDN
3. Authentication → SQL Authentication
4. Login → sqlmiadmin
5. Connect ✅

🧠 **5. Explore System Databases**

SELECT name FROM sys.databases;
You'll see:
master
model
msdb
tempdb
[YourDBs]
👉 Unlike Azure SQL Database, you have **msdb** — so SQL Agent works here!

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

📜 6. Native Backup and Restore (.bak files)

✅ Backup to Azure Blob Storage

```
-- Create a credential for Azure Storage
CREATE CREDENTIAL [https://sqlbackupstorage.blob.core.windows.net/backups]
WITH IDENTITY='SHARED ACCESS SIGNATURE',
SECRET='?sv=2024-10-09&ss=b&srt=sco&sp=rw...'; -- SAS Token

-- Backup database
BACKUP DATABASE AdventureWorks
TO URL = 'https://sqlbackupstorage.blob.core.windows.net/backups/AW.bak'
WITH COMPRESSION;
```

✅ Restore from Blob

```
RESTORE DATABASE AdventureWorks_Restore
FROM URL = 'https://sqlbackupstorage.blob.core.windows.net/backups/AW.bak'
WITH MOVE 'AdventureWorks_Data' TO 'F:\Data\AW_Restore.mdf',
    MOVE 'AdventureWorks_Log'  TO 'F:\Log\AW_Restore.ldf';
```

⚙️ 7. SQL Agent Jobs

MI supports **SQL Server Agent** fully.

Example Job (Daily Index Rebuild)

https://www.sqldbachamps.com

1. In SSMS → SQL Server Agent → Jobs → New Job
2. Name: IndexRebuild
3. Step:
4. EXEC sp_MSforeachdb
5. 'USE [?];
6. EXEC sp_updatestats;';
7. Schedule → Daily at 2 AM
8. Done ✅

🔄 8. Linked Servers & Cross-Database Queries

Cross-DB Queries:

```
SELECT * FROM [SalesDB].dbo.Orders O
JOIN [FinanceDB].dbo.Payments P ON O.OrderID = P.OrderID;
```

Linked Server Example:

```
EXEC sp_addlinkedserver
   @server='OnPremSQL',
   @srvproduct='',
   @provider='SQLNCLI',
   @datasrc='onprem-sql.mydomain.com';
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
EXEC sp_addlinkedsrvlogin
   @rmtsrvname='OnPremSQL',
   @useself='false',
   @locallogin=NULL,
   @rmtuser='sqladmin',
   @rmtpassword='P@ssword123';
```

🔒 9. Security & Network Isolation

Feature	Description
Private VNet	All MI traffic stays inside your VNet.
Public Endpoint (optional)	For external connectivity.
Managed Identity	For secure Azure resource access without passwords.
TDE / Always Encrypted / Auditing	Built-in encryption & logging.

📊 10. Performance Tuning in Managed Instance

Query Store (same as Azure SQL Database)

```
SELECT TOP 5
   qs.query_id,
   qt.query_sql_text,
   rs.avg_cpu_time,
   rs.avg_duration
FROM sys.query_store_query qs
JOIN sys.query_store_query_text qt ON qs.query_text_id = qt.query_text_id
JOIN sys.query_store_runtime_stats rs ON qs.query_id = rs.query_id
ORDER BY rs.avg_cpu_time DESC;
```

https://www.sqldbachamps.com

Index Maintenance Jobs

```
EXECUTE dbo.IndexOptimize
@Databases = 'ALL_DATABASES',
@FragmentationLow = NULL,
@FragmentationMedium = 'INDEX_REORGANIZE',
@FragmentationHigh = 'INDEX_REBUILD_ONLINE';
```

(Use Ola Hallengren's maintenance scripts — fully supported on MI)

⚡ 11. Scaling & Maintenance

Operation	Azure SQL Database	Managed Instance
Scale compute	Instantly	Few minutes downtime
Storage auto-grow	Yes	Yes
Automatic patching	Yes	Yes
Maintenance window	Configurable	Configurable

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Example PowerShell:

```
az sql mi update \
 --name sqlmi-demo2025 \
 --resource-group RG-AzureSQL-MI \
 --vcores 8
```

💰 12. Cost Optimization for Managed Instance

1. Use **Azure Hybrid Benefit** for existing SQL licenses.
2. Stop unused dev/test MI with **Pause/Resume** (via automation).
3. Right-size vCores and storage.
4. Use **General Purpose tier** for most cases; only go Business Critical if you need sub-ms latency.

🧩 13. Migration Paths to Managed Instance

Source	Tool	Description
On-Prem SQL Server	**Azure DMS (Database Migration Service)**	Automated schema & data migration
Native Backup/Restore	Use .bak via Azure Blob	Simple for one-time moves
Log Shipping / Replication	Supported	For near-zero downtime migration

📝 14. Typical DBA Use Cases in Managed Instance

Scenario	Solution
Replace On-Prem SQL	Use MI with same jobs, logins, DB Mail
Consolidate Multiple DBs	Host multiple DBs in one MI
Reporting Replica	Use Business Critical read-only replica
Secure Connectivity	Use private endpoint with ExpressRoute or VPN
Hybrid DR	Primary on-prem, secondary in MI

https://www.sqldbachamps.com

🧠 Summary: Managed Instance — Your Cloud SQL Server

Feature	Description
SQL Agent, Linked Servers	✅ Supported
Backup/Restore (.bak)	✅ Supported
Full DB Compatibility	✅ Almost 100%
Maintenance	✅ Automated
Migration	✅ Simple Lift & Shift
Security	✅ Enterprise-grade
High Availability	✅ Built-in, zone redundant

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 3: Real-Time DBA Scenarios & Troubleshooting in Azure SQL Managed Instance (MI).

⚙️ Azure SQL Managed Instance – Real-Time DBA Scenarios & Fixes

🧠 1. Scenario: High CPU Utilization in Managed Instance

🔍 **Symptoms**

- CPU > 80% consistently in Azure Portal metrics
- Users report query slowness
- Query Store shows certain queries consuming high CPU

🧪 **Diagnosis**

```
-- Top 10 CPU consuming queries in Query Store
SELECT TOP 10
  qs.query_id,
  qt.query_sql_text,
  SUM(rs.count_executions) AS [ExecutionCount],
  SUM(rs.avg_cpu_time) AS [AvgCPU],
  SUM(rs.avg_duration) AS [AvgDuration]
FROM sys.query_store_query qs
JOIN sys.query_store_query_text qt ON qs.query_text_id = qt.query_text_id
JOIN sys.query_store_runtime_stats rs ON qs.query_id = rs.query_id
GROUP BY qs.query_id, qt.query_sql_text
ORDER BY [AvgCPU] DESC;
```

🩹 **Fix**

https://www.sqldbachamps.com

1. **Check for missing indexes**

```
SELECT
  mid.statement AS TableName,
  migs.unique_compiles,
  migs.user_seeks,
  migs.avg_total_user_cost,
  'CREATE INDEX [IX_' + OBJECT_NAME(mid.object_id) + '_' +
  REPLACE(REPLACE(REPLACE(mid.equality_columns, '[', ''), ']', ''), ',', '_') + ']'
  + ' ON ' + mid.statement + '(' + ISNULL(mid.equality_columns,'')
  + ISNULL(',' + mid.inequality_columns,'') + ')'
  + ISNULL(' INCLUDE (' + mid.included_columns + ')', '') AS CreateIndexScript
FROM sys.dm_db_missing_index_group_stats AS migs
JOIN sys.dm_db_missing_index_groups AS mig ON migs.group_handle = mig.index_group_handle
JOIN sys.dm_db_missing_index_details AS mid ON mig.index_handle = mid.index_handle
ORDER BY migs.avg_total_user_cost DESC;
```

Tune high-CPU queries using **Query Store forced plan**.

Scale up vCores temporarily if required.

💾 2. Scenario: TempDB Full or Contention Issues

🔍 **Symptoms**

- Error 1101 or “Could not allocate space for object in TempDB”
- High PAGELATCH_UP waits

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧪 **Diagnosis**

```
SELECT
   SUM(user_object_reserved_page_count)*8 AS UserObjectsMB,
   SUM(internal_object_reserved_page_count)*8 AS InternalObjectsMB,
   SUM(version_store_reserved_page_count)*8 AS VersionStoreMB,
   SUM(unallocated_extent_page_count)*8 AS FreeSpaceMB
FROM sys.dm_db_file_space_usage;
```

🩹 **Fix**

1. Check open transactions:

```
SELECT
   transaction_id, name, transaction_begin_time
FROM sys.dm_tran_active_transactions;
```

2. Use **multiple TempDB data files** (configured by default in MI).
3. Avoid large sorts, hash joins, and temp tables in loops.

🧩 **3. Scenario: Blocking / Deadlocks**

🔍 **Symptoms**

- Queries hang for long time.
- sys.dm_exec_requests shows wait_type = LCK_M_IX or LCK_M_SCH_S.

🧪 **Diagnosis**

https://www.sqldbachamps.com

```
-- See current blockers
SELECT
   blocking_session_id,
   session_id,
   wait_type,
   wait_time,
   percent_complete,
   text = SUBSTRING(qt.text, (r.statement_start_offset/2)+1,
   ((CASE r.statement_end_offset
      WHEN -1 THEN DATALENGTH(qt.text)
      ELSE r.statement_end_offset
    END - r.statement_start_offset)/2) + 1)
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS qt
WHERE blocking_session_id <> 0;
```

🩹 **Fix**

- Identify blocker and kill it (if safe):
- KILL <blocking_session_id>;
- Apply **NOLOCK** or **READ COMMITTED SNAPSHOT** isolation.
- Create missing indexes to reduce locking duration.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🚦 4. Scenario: Query Slow after Migration to Managed Instance

🔍 **Symptoms**

- Query execution plans differ after moving from on-prem.
- Parameter sniffing or cardinality estimation issues.

🧪 **Diagnosis**

```
SELECT last_compile_time, execution_count, plan_handle
FROM sys.dm_exec_cached_plans cp
CROSS APPLY sys.dm_exec_query_plan(plan_handle) qp
WHERE cp.objtype = 'Prepared';
```

🩹 **Fix**

1. Add **OPTION (RECOMPILE)** for sensitive parameters.
2. Use **OPTIMIZE FOR UNKNOWN** in procedures.
3. Compare execution plan with on-prem using SET SHOWPLAN_XML ON.

📊 5. Scenario: High IO Latency in Managed Instance

🔍 **Symptoms**

- Long waits on PAGEIOLATCH_XX
- High read/write latency visible in Azure Metrics

🧪 **Diagnosis**

```
SELECT
  DB_NAME(database_id) AS DBName,
  file_id,
  io_stall_read_ms / num_of_reads AS ReadLatency_ms,
  io_stall_write_ms / num_of_writes AS WriteLatency_ms
FROM sys.dm_io_virtual_file_stats(NULL, NULL);
```

https://www.sqldbachamps.com

🩹 **Fix**

- Switch to **Business Critical** tier for local SSD performance.
- Add **appropriate indexes** and partition large tables.
- Enable **Read Scale-Out** for reporting workloads.

🔐 6. Scenario: Login or Connectivity Issues

🔍 **Symptoms**

- Cannot connect via SSMS
- Error 18456 (Login failed for user)
- “Cannot open server requested by login.”

🩹 **Fix**

1. Validate Public Endpoint configuration.
2. Add client IP in **Firewall Rules**.
3. Ensure correct **port 3342**.
4. Check user exists:
5. SELECT name, type_desc FROM sys.sql_logins;
6. If using **AAD Authentication**, assign proper roles.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

💣 7. Scenario: Deadlocks Captured in MI

Managed Instance supports **Extended Events** to capture deadlocks.

🧪 **Setup**

```
CREATE EVENT SESSION [DeadlockCapture] ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
ADD TARGET package0.ring_buffer
WITH (STARTUP_STATE=ON);
ALTER EVENT SESSION [DeadlockCapture] ON SERVER STATE = START;
```

🧩 **Read Deadlock Details**

```
SELECT CAST(event_data AS XML) AS DeadlockGraph
FROM sys.fn_xe_file_target_read_file('system_health*.xel', NULL, NULL, NULL)
WHERE event_data LIKE '%deadlock%';
```

🔍 8. Scenario: Detecting Long-Running Queries

```
SELECT
  s.session_id,
  r.status,
  r.cpu_time,
  r.total_elapsed_time/1000 AS Duration_ms,
  SUBSTRING(t.text, (r.statement_start_offset/2)+1,
    ((CASE r.statement_end_offset
      WHEN -1 THEN DATALENGTH(t.text)
      ELSE r.statement_end_offset END
      - r.statement_start_offset)/2)+1) AS running_query
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.status NOT IN ('background','sleeping')
ORDER BY r.total_elapsed_time DESC;
```

https://www.sqldbachamps.com

💾 9. Scenario: Slow Backup or Restore

🧪 **Diagnosis**

- Backup to Blob very slow (network bottleneck)
- Check throughput via Azure Metrics

🩹 **Fix**

- Use **page blob storage** (not block blob).
- Enable **compression** in backup.
- Split large backups into multiple files:
- BACKUP DATABASE AdventureWorks
- TO
- URL='https://storage.blob.core.windows.net/backups/AW1.bak',
- URL='https://storage.blob.core.windows.net/backups/AW2.bak'
- WITH COMPRESSION, STATS=10;

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

📜 10. Scenario: Query Store Full or Disabled

🧪 **Diagnosis**

```
SELECT actual_state_desc, readonly_reason
FROM sys.database_query_store_options;
```

🩹 **Fix**

```
ALTER DATABASE [YourDB] SET QUERY_STORE CLEAR;
ALTER DATABASE [YourDB] SET QUERY_STORE (OPERATION_MODE = READ_WRITE);
```

🔄 11. Scenario: Database Auto-Growth & Storage Monitoring

🧪 **Diagnosis**

```
SELECT
  DB_NAME(database_id) AS DBName,
  file_id, type_desc, size*8/1024 AS SizeMB, max_size
FROM sys.database_files;
```

🩹 **Fix**

- Enable **autogrow** with sensible increments:
- ALTER DATABASE [YourDB]
- MODIFY FILE (NAME = N'YourDB_Data', FILEGROWTH = 512MB);
- Monitor storage in Azure Metrics regularly.

🧩 12. Scenario: Identify Wait Statistics

🧪 **Diagnosis**

https://www.sqldbachamps.com

```
SELECT TOP 15
  wait_type,  wait_time_ms/1000.0 AS WaitTime_s,
  signal_wait_time_ms/1000.0 AS SignalWait_s,
  waiting_tasks_count
FROM sys.dm_os_wait_stats
WHERE wait_type NOT LIKE 'SLEEP%'
ORDER BY wait_time_ms DESC;
```

🧩 **Interpretation**

Wait Type	Root Cause
PAGEIOLATCH	IO bottleneck
CXPACKET	Parallelism imbalance
LCK_M_XX	Lock contention
SOS_SCHEDULER_YIELD	CPU pressure

🧠 Bonus: Real-World MI DBA Checklist

✅ Monitor CPU, IO, TempDB, Storage, Query Store
✅ Keep Query Store trimmed (CLEANUP_POLICY)
✅ Configure Alerts for long running queries
✅ Backup to Azure Blob daily
✅ Automate index rebuild via SQL Agent
✅ Regularly review sys.dm_exec_query_stats

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 4: Azure SQL Managed Instance (MI) – Backup, Restore, Disaster Recovery & High Availability (HA) Scenarios.

This is one of the most critical topics for **real-time SQL Cloud DBAs**, especially if you manage production workloads in **Managed Instance** where uptime, RPO (Recovery Point Objective), and RTO (Recovery Time Objective) matter most.
Let's go step by step 👇

📦 1. Azure SQL Managed Instance – Backup & Restore Overview

Azure SQL MI **automatically manages backups** for you — no need to manually schedule full/diff/log backups unless you want custom copies.

Backup Type	Frequency	Retention	Storage Location
Full	Weekly	Default 7–35 days	Azure Storage (managed by Microsoft)
Differential	Every 12 hours	Same	Managed by Microsoft
Transaction Log	Every 5–10 minutes	Same	Managed by Microsoft

✅ You don't manage .bak files manually (unless for migration).
✅ You can perform **Point-in-Time Restore** (PITR) at the database level.

💾 2. Point-in-Time Restore (PITR)

🎯 **Use Case:**
Your app team accidentally dropped a table or updated 1M rows wrongly.

🧩 **Solution:**
Restore DB to a specific point before failure.

https://www.sqldbachamps.com

Steps (Azure Portal):

1. Go to **Azure Portal → SQL Managed Instance → Databases → yourDB**
2. Click **Restore**
3. Choose:
   - **Restore point in time:** e.g., *2025-10-09 02:15:00 UTC*
   - **New database name:** yourDB_Restore_20251009
4. Click **OK** ✅

Azure creates a new DB within the same MI.

🔍 **Using PowerShell**

```
$restorePoint = Get-Date "2025-10-09T02:15:00Z"
Restore-AzSqlInstanceDatabase `
  -FromPointInTimeBackup `
  -ResourceGroupName "RG-SQLMI" `
  -InstanceName "sqlmi-demo2025" `
  -Name "yourDB" `
  -TargetDatabaseName "yourDB_Restore_20251009" `
  -PointInTime $restorePoint
```

⏱️ Takes a few minutes depending on DB size.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

☁️ 3. Geo-Restore (Region-Level DR)

🎯 **Use Case:**

Your entire Azure region (e.g., East US) becomes unavailable.

🧩 **Solution:**

Azure keeps **geo-redundant backups** (if enabled). You can restore your DB to any **other region**.

🪜 **Steps:**

1. Go to **SQL Managed Instance → Databases → Restore**
2. Choose **Restore to different region**
3. Pick the **geo-paired region** (e.g., West US if East US fails)
4. Click **Create** ✅

RTO: ~1–2 hours

RPO: ≤1 hour (based on log backup interval)

🔁 4. Native Backup and Restore (.bak) with Azure Blob

✅ Supported in Managed Instance (unlike Azure SQL Database).

📤 **Backup to Azure Blob**

```
-- Create credential
CREATE CREDENTIAL [https://sqlbackupstorage.blob.core.windows.net/backups]
WITH IDENTITY='SHARED ACCESS SIGNATURE',
SECRET='?sv=2025-10-09&ss=b&srt=sco&sp=rw...';

-- Backup database
BACKUP DATABASE AdventureWorks
TO URL='https://sqlbackupstorage.blob.core.windows.net/backups/AW_Full.bak'
WITH COMPRESSION, STATS=10;
```

https://www.sqldbachamps.com

📥 **Restore from Azure Blob**

```
RESTORE DATABASE AdventureWorks_Restore
FROM URL='https://sqlbackupstorage.blob.core.windows.net/backups/AW_Full.bak'
WITH MOVE 'AdventureWorks_Data' TO 'F:\Data\AW_Data.mdf',
   MOVE 'AdventureWorks_Log'  TO 'F:\Log\AW_Log.ldf';
```

🧩 5. Managed Instance High Availability (Built-In)

💡 **Concept:**

Each Managed Instance comes with **automatic HA** using an **Always On availability group** managed internally by Azure.

📦 **Architecture:**

Component	Description
Primary Replica	Handles reads/writes
Secondary Replica	Auto-failover within region
Synchronization	Synchronous (Business Critical) / Asynchronous (General Purpose)
Failover Time	~10–30 seconds (automatic)

You don't see this configuration — Azure manages it behind the scenes.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

💼 6. Auto-Failover Groups (Cross-Region HA & DR)

🎯 **Use Case:**

Your org requires **automatic failover across Azure regions** for business continuity.

🧩 **Solution:**

Use **Auto-Failover Groups (AFG)** — replicate databases between two Managed Instances in different regions.

Steps to Configure Auto-Failover Group

Step 1. Prepare Two Managed Instances:

Role	Region	Name
Primary	East US	sqlmi-prod-eastus
Secondary	West US	sqlmi-dr-westus

Step 2. Create Failover Group (Azure Portal)

1. Go to **Primary MI → Failover Groups → Add Group**
2. Group name: FailoverGroup-ProdDR
3. Select secondary MI
4. Choose databases to include
5. Enable **Automatic Failover**
6. Click **Create**

🔗 **Connection String**

Azure provides two endpoints automatically:

Endpoint Type	Example
Primary	FailoverGroup-ProdDR.database.windows.net
Secondary (read-only)	FailoverGroup-ProdDR.secondary.database.windows.net

https://www.sqldbachamps.com

Your app connects using the **primary endpoint**, and it automatically redirects after failover.

🔄 **Manual Failover via PowerShell**

```
Switch-AzSqlDatabaseInstanceFailoverGroup `
  -Name "FailoverGroup-ProdDR" `
  -ResourceGroupName "RG-Prod" `
  -ServerName "sqlmi-prod-eastus"
```

🧠 **7. Failover Testing & Validation**

✅ **Verify Replication Status**

```
SELECT * FROM sys.dm_geo_replication_link_status;
```

✅ **Test Failover**

Temporarily switch to DR region using Azure Portal → Failover Group → **Failover Now**.

✅ **Validate**

Run:

```
SELECT @@SERVERNAME, SYSDATETIME();
```

You'll see DR instance name confirming successful failover.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

📑 8. Database Copy between Managed Instances

You can **copy** databases between MIs in same or different regions.

Example:

```
CREATE DATABASE [SalesDB_Copy]
AS COPY OF [SalesDB];
```

or via **Portal → Database → Copy**.

🧮 9. Backup Retention Policy

You can change backup retention up to **35 days** (default: 7 days).

📜 **PowerShell:**

```
Set-AzSqlInstanceDatabaseBackupShortTermRetentionPolicy `
  -ResourceGroupName "RG-SQLMI" `
  -InstanceName "sqlmi-demo2025" `
  -DatabaseName "SalesDB" `
  -RetentionDays 30
```

🧰 10. Disaster Recovery Plan – Real-World Template

Step	Action	Owner	RTO	RPO
1	Enable auto-failover group	DBA	5 mins	<1 hour
2	Test failover quarterly	DBA	N/A	N/A
3	Enable geo-backup	Cloud Admin	1 hour	1 hour
4	Configure alerts for failure	DBA	2 mins	N/A
5	Maintain PITR documentation	DBA	N/A	N/A
6	Validate app connectivity post-failover	App Team	10 mins	N/A

https://www.sqldbachamps.com

⚡ 11. Real-Time DBA Commands (HA & DR)

Check Failover Mode:

```
SELECT * FROM sys.failover_groups;
```

View Geo-Replica Status:

```
SELECT
   partner_server,
   role_desc,
   replication_state_desc,
   last_replication,
   link_failover_mode_desc
FROM sys.dm_geo_replication_link_status;
```

Force Manual Failover:

```
ALTER FAILOVER GROUP [FailoverGroup-ProdDR] FORCE_FAILOVER_ALLOW_DATA_LOSS;
```

(Use only in DR testing or emergency.)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

12. Summary – Managed Instance Backup & HA Key Takeaways

Topic	Description
Automatic Backups	Full + Diff + Log every few mins
PITR	Restore to any point in 7–35 days
Geo-Backups	Restore in other region if primary region fails
Native .bak	Supported to/from Blob
Built-In HA	Always On replication managed by Azure
Auto-Failover Groups	Cross-region DR with auto-failover
RPO	≤ 1 hour
RTO	Minutes to hour

https://www.sqldbachamps.com

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 5: Azure SQL Managed Instance – Security, Auditing & Compliance Deep Dive.

This is where you learn to **protect, monitor, and harden** your Managed Instance (MI) against insider and external threats while meeting compliance requirements (ISO, SOC, GDPR, etc.).

🧱 1. Azure SQL Managed Instance Security Model – Overview

Security Layer	Purpose	Who Manages
Physical / Infrastructure	Azure datacenter, VMs, storage	Microsoft
Network Security	VNet, NSG, Private Endpoint	You
Authentication & Access	SQL logins, Azure AD, RBAC	You
Data Encryption	TDE, TLS, Always Encrypted	You
Auditing & Threat Detection	Logging, alerts	You

Managed Instance follows **Shared Responsibility Model** — Microsoft secures the infra, you secure configuration & data.

🌐 2. Network Security in Managed Instance

✅ Managed Instance is **always deployed inside a private VNet** — isolated from public internet.
You can optionally enable a **public endpoint** for SSMS or app access, but by default it's private.

⚙️ VNet Configuration

- Each MI subnet must be *dedicated* (no other resource allowed).
- Use **Network Security Groups (NSGs)** to control inbound/outbound traffic.
- Use **VPN / ExpressRoute** for on-prem access.

🧩 Example NSG Rule

https://www.sqldbachamps.com

Rule	Source	Destination	Port	Action
Allow SQL MI Access	On-prem network	MI subnet	1433, 3342	Allow
Deny Internet Access	*	*	Any	Deny

📜 3. Authentication in Managed Instance

Managed Instance supports **three authentication modes**:

Authentication Type	Description
SQL Authentication	Login with username & password (default)
Windows (AAD) Authentication	Login via Azure Active Directory user/group
Managed Identity	Use MI's own identity to connect to Azure resources securely

🧩 3.1 SQL Authentication

Classic method — create SQL login manually.

```
CREATE LOGIN [appuser] WITH PASSWORD = 'Str0ngP@ss!';
CREATE USER [appuser] FOR LOGIN [appuser];
ALTER ROLE db_datareader ADD MEMBER [appuser];
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧩 3.2 Azure AD Authentication

Enables passwordless, centralized identity.

🧭 **Steps to Enable AAD Auth:**

1. In Azure Portal → MI → **Active Directory Admin**
   → Set admin as your AAD user (e.g., *praveen@yourdomain.com*).
2. Connect via SSMS → Authentication: **Azure Active Directory - Universal with MFA**.
3. Create database users:
   CREATE USER [praveen@yourdomain.com] FROM EXTERNAL PROVIDER;
   ALTER ROLE db_datareader ADD MEMBER [praveen@yourdomain.com];

✅ AAD groups are supported too:
CREATE USER [DBA-Team] FROM EXTERNAL PROVIDER;

🧩 3.3 Managed Identity (MI Identity)

Your Managed Instance has its own **service principal in AAD** — used for secure access to other Azure resources (e.g., Key Vault, Blob).
Example: Access Azure Blob securely from MI without keys.
CREATE DATABASE SCOPED CREDENTIAL [AzureBlobCred]
WITH IDENTITY = 'Managed Identity';

🔐 4. Data Encryption in Managed Instance

https://www.sqldbachamps.com

There are three main layers of encryption:

Type	Purpose	Default	Configurable
TDE (Transparent Data Encryption)	Encrypts data/log files at rest	✅ Yes	Customer key optional
Always Encrypted	Protects sensitive columns (e.g., SSN)	❌ No	Must configure manually
TLS (Encryption in Transit)	Encrypts network communication	✅ Yes	Auto-managed

🧩 4.1 Transparent Data Encryption (TDE)

✅ Enabled by default in MI.
TDE encrypts the entire DB using a **Database Encryption Key (DEK)** stored in a **service-managed certificate**.

Check TDE Status:

```
SELECT db_name(database_id) AS DBName, encryption_state, key_algorithm, key_length
FROM sys.dm_database_encryption_keys;
```

Example Output:

DBName	encryption_state	key_algorithm	key_length
SalesDB	3 (Encrypted)	AES_256	256

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧩 4.2 Customer-Managed Key (Bring Your Own Key – BYOK)

You can store the TDE protector in **Azure Key Vault** instead of letting Microsoft manage it.

🔑 **Steps:**

1. Create Azure Key Vault and import your key.
2. Grant MI's Managed Identity **Key Vault Crypto Officer** access.
3. Configure via PowerShell:
4. Set-AzSqlInstanceTransparentDataEncryptionProtector `
5. -ResourceGroupName "RG-MI" `
6. -InstanceName "sqlmi-prod" `
7. -Type AzureKeyVault `
8. -KeyId "https://yourkeyvault.vault.azure.net/keys/MyTDEKey/abc123"

✅ Benefit: Full control + key rotation for compliance (e.g., HIPAA, PCI-DSS).

🧠 5. Always Encrypted (Column-Level Encryption)

Encrypt sensitive data like SSN, PAN, or salary — even DBAs cannot see decrypted values.

Example:

```
CREATE COLUMN MASTER KEY [CMK_AzureKeyVault]
WITH KEY_STORE_PROVIDER_NAME = N'AZURE_KEY_VAULT',
KEY_PATH = N'https://yourvault.vault.azure.net/keys/CustomerKey/';

CREATE COLUMN ENCRYPTION KEY [CEK_1]
WITH VALUES (COLUMN_MASTER_KEY = [CMK_AzureKeyVault], ALGORITHM = 'RSA_OAEP');

CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  SSN NVARCHAR(11) COLLATE Latin1_General_BIN2 ENCRYPTED WITH (
    COLUMN_ENCRYPTION_KEY = [CEK_1],
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256') NOT NULL,
  Name NVARCHAR(50)
);
```

https://www.sqldbachamps.com

⚠️ Encryption/decryption happens on the client side (via .NET driver).

📜 6. SQL Auditing

Audit all database and login-level activities to **Azure Storage**, **Event Hub**, or **Log Analytics**.

Steps to Enable Auditing:

1. Go to **SQL Managed Instance → Auditing**
2. Set **Auditing: ON**
3. Destination: *Log Analytics Workspace* or *Storage Account*
4. Optionally enable **Audit log retention** (e.g., 90 days)

View Logs:

- Portal → **Monitor → Logs → AuditLogs**
- Or query via Kusto (Log Analytics):

- AzureDiagnostics
- | where Category == "SQLSecurityAuditEvents"
- | where LogicalServerName_s == "sqlmi-demo2025"
- | project TimeGenerated, statement_s, succeeded_s

🧰 7. Advanced Threat Protection (ATP)

Built-in AI system that monitors suspicious DB behavior.

Threat Type	Example Detection
SQL Injection	Unexpected query patterns
Privilege Escalation	Role/permission changes
Data Exfiltration	Large data exports
Brute Force	Multiple failed login attempts

Enable via:

Azure Portal → MI → **Microsoft Defender for SQL** → Enable.

When triggered, you get email alerts + detailed investigation logs.

🧮 8. Row-Level Security (RLS)

Control which rows users can see based on their identity.

Example:

```
CREATE FUNCTION dbo.fn_SecurityPredicate(@UserName AS sysname)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_result
WHERE @UserName = USER_NAME();

CREATE SECURITY POLICY EmployeeFilter
ADD FILTER PREDICATE dbo.fn_SecurityPredicate(USER_NAME())
ON dbo.Employees
WITH (STATE = ON);
```

https://www.sqldbachamps.com

✅ Each user sees only their own rows.

🔍 9. Dynamic Data Masking (DDM)

Mask sensitive columns for non-privileged users.

```
ALTER TABLE Customers
ALTER COLUMN CreditCardNumber ADD MASKED WITH (FUNCTION = 'partial(0,"XXXX-XXXX-XXXX-",4)');
```

Now, users without UNMASK permission see masked data like XXXX-XXXX-XXXX-1234.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧩 10. Security Best Practices Checklist

Area	Best Practice
Authentication	Use AAD over SQL logins
Encryption	Enable TDE + Always Encrypted for sensitive columns
Backup	Store TDE keys in Key Vault
Auditing	Send logs to Log Analytics
Threat Detection	Enable Defender for SQL
Network	Disable Public Endpoint unless absolutely needed
Patching	Automatic – no manual patching
Privileges	Follow Least Privilege principle (no sysadmin logins to apps)

🧪 11. Quick Verification Scripts

Check Encryption Status

```
SELECT db_name(database_id), encryption_state
FROM sys.dm_database_encryption_keys;
```

Check Auditing Status

```
SELECT * FROM sys.database_audit_specifications;
```

Check TDE Protector Type

```
SELECT * FROM sys.dm_database_encryption_keys;
```

Check AAD Login

```
SELECT name, type_desc FROM sys.database_principals WHERE authentication_type_desc = 'EXTERNAL';
```

https://www.sqldbachamps.com

🧠 12. Summary – Managed Instance Security in a Nutshell

Security Feature	Purpose	Default
VNet Isolation	Network boundary	✅
TDE	Encrypts DB at rest	✅
AAD Auth	Centralized login control	Optional
Managed Identity	Secure Azure resource access	✅
Auditing	Tracks activity logs	Optional
ATP	Detects suspicious behavior	Optional
RLS / DDM	Granular data visibility control	Optional
BYOK / CMK	Customer-controlled keys	Optional

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 6: Azure SQL Managed Instance Monitoring, Alerts & Performance Optimization

Here you'll master **real-time monitoring**, **query performance tuning**, and **automated alerting** — all cloud-native, no agents needed.

🧱 1. Why Monitoring Matters in Azure SQL MI

Even though Azure manages the hardware, **you're responsible for performance** — slow queries, blocking, CPU spikes, memory pressure, etc.

Your goals as a DBA:

- Detect performance degradation early
- Capture query trends over time
- Set automated alerts
- Tune queries & indexes proactively

Azure gives three monitoring layers:

Level	Tool	Use
Basic Metrics	Azure Portal → Metrics	Quick view (CPU, storage, sessions)
Advanced Diagnostics	Log Analytics / Azure Monitor	Deep troubleshooting
Query-Level	Query Store, DMVs, Extended Events	Root cause analysis

⚙️ 2. Basic Metrics Monitoring (Portal)

Open: **Azure Portal → SQL Managed Instance → Monitoring → Metrics**

You'll see built-in metrics such as:

Metric	Description
CPU Percent	CPU usage of MI
IO Bytes Read/Write	IOPS throughput
Storage Space Used	DB size trend
Sessions Count	Active sessions
DTU / vCore Utilization	Compute performance baseline

https://www.sqldbachamps.com

✅ **Pro Tip:**

Set a *chart with multiple metrics* — CPU + IO + Sessions — to detect correlation during incidents.

🚨 3. Creating Metric Alerts

Example: Alert on CPU > 80% for 10 mins

🧭 Steps:

1. Azure Portal → SQL Managed Instance → Alerts → **+ Create Alert Rule**
2. Signal Name → CPU percent
3. Condition → Greater than 80
4. Frequency → 5 minutes
5. Action Group → Send Email/SMS/Webhook to DBA Team
6. Save ✅

You'll now receive automatic alerts when sustained CPU pressure occurs.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

📊 4. Diagnostic Settings (Log Analytics Integration)

To capture **query-level details, blocking, waits, and deadlocks**, enable diagnostics.

⚙️ Steps:

1. Azure Portal → MI → **Diagnostic Settings** → **+ Add Diagnostic Setting**
2. Categories to enable:
   - SQLInsights
   - AutomaticTuning
   - SQLSecurityAuditEvents
   - Errors
   - Timeouts
   - Blocks
3. Destination → **Log Analytics Workspace**

📈 5. Querying Data from Log Analytics

Once integrated, you can run **Kusto queries** in Log Analytics.

Example 1: Top 10 CPU-consuming Queries

```
AzureDiagnostics
| where Category == "SQLInsights"
| where ResourceType == "MANAGEDINSTANCE"
| summarize AvgCPU = avg(cpu_percent) by query_hash_s
| top 10 by AvgCPU desc
```

https://www.sqldbachamps.com

Example 2: Top Wait Types

```
AzureDiagnostics
| where Category == "SQLWaitStats"
| summarize TotalWaitTime = sum(wait_time_ms) by wait_type_s
| top 10 by TotalWaitTime desc
```

🧠 6. Query Store – Built-in Performance Repository

Query Store in MI works *exactly like on-prem SQL Server* — tracks **query text, plan, runtime stats**.

⚙️ Enable Query Store:

```
ALTER DATABASE [SalesDB] SET QUERY_STORE = ON;
ALTER DATABASE [SalesDB]
SET QUERY_STORE (OPERATION_MODE = READ_WRITE, CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30));
```

View High-CPU Queries:

```
SELECT TOP 10
   qs.query_id, qt.query_sql_text, rs.avg_cpu_time, rs.count_executions
FROM sys.query_store_query_text qt
JOIN sys.query_store_query qs ON qt.query_text_id = qs.query_text_id
JOIN sys.query_store_runtime_stats rs ON qs.query_id = rs.query_id
ORDER BY rs.avg_cpu_time DESC;
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✅ Use Query Store to:

- Compare performance *before and after* deployments
- Identify **regressed queries**
- Force the best plan:

```
EXEC sp_query_store_force_plan @query_id = 101, @plan_id = 45;
```

🧩 7. Extended Events (XEvents) – Deep Diagnostics

When you need precise detail (blocking chains, deadlocks, wait events), use **Extended Events**.

Example: Capture Deadlocks

```
CREATE EVENT SESSION [Deadlocks]
ON SERVER
ADD EVENT sqlserver.lock_deadlock
ADD TARGET package0.event_file (SET filename = 'deadlocks.xel');
ALTER EVENT SESSION [Deadlocks] ON SERVER STATE = START;
```

View results:

```
SELECT event_data = CONVERT(XML, event_data)
FROM sys.fn_xe_file_target_read_file('deadlocks*.xel', NULL, NULL, NULL);
```

🧰 8. DMVs for Live Monitoring

Azure SQL Managed Instance supports most on-prem DMVs.

Area	DMV	Description
Sessions	sys.dm_exec_sessions	Current logins/sessions
Requests	sys.dm_exec_requests	Active queries
Waits	sys.dm_os_wait_stats	Resource waits
IO	sys.dm_io_virtual_file_stats	File IO latency
Memory	sys.dm_os_memory_clerks	Memory usage
Index	sys.dm_db_index_usage_stats	Index efficiency

https://www.sqldbachamps.com

Example: Top CPU Queries

```
SELECT TOP 10
  total_worker_time/1000 AS CPU_ms, execution_count,
  total_worker_time/execution_count AS Avg_CPU,
  SUBSTRING(qt.text, 1, 200) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY CPU_ms DESC;
```

🧮 9. Automatic Tuning

Managed Instance can automatically:

- **Force Last Known Good Plan**
- **Create Missing Indexes**
- **Drop Unused Indexes**

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Enable It:

Azure Portal → MI → **Automatic tuning → Configure**

Or via T-SQL:

ALTER DATABASE [SalesDB] SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);

✅ Use this in test/staging first to validate before enabling in production.

📈 10. Performance Optimization Scenarios

Scenario 1: High CPU

- Check Query Store → Identify expensive queries
- Use Query Tuning Advisor or missing index DMVs
- Example:
- SELECT * FROM sys.dm_db_missing_index_details;

Scenario 2: High IO / Slow Storage

- Check sys.dm_io_virtual_file_stats
- Move to **Premium Storage tier** if latency > 10ms.

Scenario 3: Blocking

- Use:
- SELECT blocking_session_id, wait_type, wait_time, text
- FROM sys.dm_exec_requests CROSS APPLY sys.dm_exec_sql_text(sql_handle)
- WHERE blocking_session_id <> 0;
- Capture via Extended Events (Deadlock graph).

⚙️ 11. Performance Baselining Strategy

Always maintain a **baseline** for normal behavior:

https://www.sqldbachamps.com

- Avg CPU: < 60%
- Avg IO Latency: < 10 ms
- Query Duration: < 2 sec
- Blocking count: < 5

Store historical baselines in **Log Analytics or Excel dashboard** for quick comparison during incidents.

🔔 12. Automation with Azure Monitor

Integrate with:

- **Azure Automation Runbooks** → Auto-scale, restart, or notify.
- **Power BI** → Visual dashboards.
- **ServiceNow / Teams / Slack** → Alert notifications.

✅ Example: Trigger a Power Automate flow when CPU > 90% for 15 minutes.

🧮 13. Example Daily Monitoring Checklist (for DBAs)

Task	Tool	Frequency
Check CPU/IO/Storage metrics	Azure Portal	Daily
Review Query Store regressed queries	SSMS	Daily
Verify blocking/deadlocks	XEvents / DMV	Daily
Review auto-tuning recommendations	Portal	Weekly
Review audit/threat alerts	Defender for SQL	Weekly
Validate backups & DR sync	Portal	Weekly

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧠 14. Summary – Managed Instance Monitoring in a Nutshell

Area	Tool	Goal
Real-time metrics	Azure Monitor	CPU, IO, storage
Alerts	Azure Alerts	Notify issues
Query-level	Query Store	Plan analysis
Deep diagnostics	XEvents, DMVs	Root cause
Auto optimization	Automatic Tuning	Self-healing
Visualization	Power BI / Log Analytics	Trend analysis

✅ You now know how to **monitor, diagnose, and optimize** performance in Azure SQL Managed Instance like an enterprise-grade DBA.

https://www.sqldbachamps.com

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 7 (Version-1):

High Availability (HA), Backups, and Disaster Recovery (DR) in Azure SQL Managed Instance — the *lifeline* of enterprise-grade SQL deployments.

By the end of this module, you'll know exactly how to configure, monitor, and validate business continuity solutions for your SQL Managed Instances — **zero data loss, minimum downtime** 🚀

🧱 1. Azure SQL Managed Instance – Built-In High Availability Architecture

Azure SQL Managed Instance is designed with **99.99% uptime SLA**, thanks to its **built-in Always On Availability Group architecture** (you don't manage WSFC or replicas manually).

🔧 **Internal Architecture:**

- **Primary Replica** – Handles read/write workload.
- **Secondary Replica(s)** – Used for HA failover and automatic backups.
- **Automatic failover** within the same region.

💡 **Key Facts:**

Feature	Managed By	Description
Always On AG	Microsoft	Multi-replica setup for fault tolerance
Synchronous Commit	Yes	For high availability
Automatic Failover	Yes	Within same region
Transparent to Apps	Yes	Connection retry handles failover
Log Transport	Managed by Azure	No manual setup

https://www.sqldbachamps.com

🧭 2. Understanding Azure SQL MI Redundancy Models

There are **two redundancy types** that define how your MI's underlying storage and replicas are configured.

Redundancy	Description	Recommended For
Zone Redundant (ZRS)	Replicas across multiple Availability Zones (same region)	Mission-critical workloads
Locally Redundant (LRS)	Replicas within single zone	Cost-effective non-critical apps

⚙️ **How to Configure:**

When creating a Managed Instance:

High Availability → Zone Redundant = Enabled ✅

Once deployed, you can't switch redundancy type; it's fixed at creation.

🗓️ 3. Built-In Automated Backups

Azure automatically performs:

- **Full backup**: Weekly
- **Differential backup**: Every 12–24 hours
- **Transaction log backup**: Every 5–10 minutes

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🗓️ Backup Retention:

Edition	Retention	Max Retention
General Purpose	7–35 days	35 days
Business Critical	7–35 days	35 days
Long-term Backup Retention (LTR)	Up to 10 years	Customizable

💾 4. Point-in-Time Restore (PITR)

You can restore your Managed Instance database to any specific time within retention window.

🧭 Steps via Portal:

Azure Portal → SQL Managed Instance → Databases → Select DB → **Restore**
→ Choose **Point in Time** → Pick Date/Time → Restore.

👨‍💻 T-SQL Example:

```
-- Restore SalesDB to a specific time
RESTORE DATABASE [SalesDB_Restore_2025]
FROM DATABASE_SNAPSHOT = 'SalesDB'
AS OF '2025-10-09T08:30:00';
```

✅ Azure creates a new database (same instance) with data as of that exact time.

🧰 5. Long-Term Retention (LTR) – Archiving Backups up to 10 Years

LTR keeps selected full backups for years in Azure Blob storage (for compliance).

https://www.sqldbachamps.com

🧭 Enable LTR:

1. Portal → MI → **Backups → Long-term retention**
2. Policy Example:
   - Weekly: Retain for 12 weeks
   - Monthly: Retain for 12 months
   - Yearly: Retain for 10 years

Or via PowerShell:

```
Set-AzSqlDatabaseBackupLongTermRetentionPolicy `
  -ResourceGroupName "RG-SQL" `
  -ServerName "sqlmi-prod" `
  -DatabaseName "SalesDB" `
  -WeeklyRetention "P12W" `
  -MonthlyRetention "P12M" `
  -YearlyRetention "P10Y"
```

🌍 6. Geo-Replication – Cross-Region Disaster Recovery

To protect against regional outages, use **Auto-Failover Groups (AFG)**.
They replicate databases asynchronously across **two Managed Instances in different regions**.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

💡 **Features:**

Feature	Description
Async replication	Between primary & secondary MI
Readable secondary	Yes
Auto-failover	Yes
Connection string	Single endpoint for apps
RPO	< 5 sec typical
RTO	< 60 sec

7. Setting Up Auto-Failover Group (Portal)

1. Deploy **two Managed Instances** in different regions (e.g., East US & West US).
2. Primary MI → **Failover Groups → Add Group**
3. Configure:
   - Secondary MI name
   - Group name → SQLMI-FG-PROD
   - Databases to include
   - Enable **Automatic failover**
4. Azure creates:
   - **Primary endpoint:** sqlmi-primary.database.windows.net
   - **Secondary endpoint:** sqlmi-secondary.database.windows.net

✅ Use these endpoints in app connection strings for transparent failover.

https://www.sqldbachamps.com

💻 8. PowerShell Example: Create Auto-Failover Group

```
New-AzSqlDatabaseFailoverGroup `
  -ResourceGroupName "RG-Prod" `
  -ServerName "sqlmi-eastus" `
  -PartnerServerName "sqlmi-westus" `
  -FailoverGroupName "FG-MI-PROD" `
  -FailoverPolicy Automatic `
  -GracePeriodWithDataLossHours 1 `
  -AddDatabase "SalesDB","FinanceDB"
```

Verify:

```
Get-AzSqlDatabaseFailoverGroup -ResourceGroupName "RG-Prod" -ServerName "sqlmi-eastus"
```

🔁 9. Manual Failover (Planned or DR Drill)

To simulate or perform DR switch:

Portal:

MI → Failover Groups → Select Group → **Failover**

PowerShell:

```
Switch-AzSqlDatabaseFailoverGroup `
  -ResourceGroupName "RG-Prod" `
  -ServerName "sqlmi-eastus" `
  -FailoverGroupName "FG-MI-PROD"
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

✅ Use this during maintenance windows or DR drills.

📈 10. Monitoring HA & DR Health

Portal Metrics:

- replica_role – Primary/Secondary
- replication_lag_sec – Data sync delay
- failover_count – Failover occurrences

Query Example:

```
SELECT role_desc, synchronization_state_desc, database_id
FROM sys.dm_hadr_database_replica_states;
```

✅ Should show SYNCHRONIZED or SYNCHRONIZING.

⚙️ 11. Backup Validation (DBCC CHECKDB)

Even though Azure manages backups, *you must validate restores regularly.*

Sample Validation Process:

1. Restore backup to non-prod MI.
2. Run:
3. DBCC CHECKDB ('SalesDB_Restore') WITH NO_INFOMSGS;
4. Validate integrity logs and retention policies.

🧮 12. Performance Impact Considerations

Area	Notes
Backup IO	Offloaded to secondary replica
PITR Restore	Creates new DB (storage usage ↑)
Auto-Failover	App reconnection delay ~30–60 sec
Geo-Replication	Asynchronous (possible minor data loss < 5 sec)

https://www.sqldbachamps.com

🔒 13. Security & Compliance for Backups

✅ All backups are:

- Encrypted using **TDE**.
- Stored securely in **RA-GRS (geo-redundant)** storage.
- Accessible only to **Azure fabric**, not users.

You can bring your own TDE Key (BYOK) via **Azure Key Vault** for compliance.

🧰 14. Disaster Recovery Drill Checklist

Step	Task	Validation
1	Trigger manual failover	Successful promotion
2	App connection test	Connection string resolves new primary
3	Data validation	Row count match between replicas
4	Performance test	No query latency issues
5	Failback	Restore original region as primary

✅ Schedule quarterly DR drills to ensure RPO/RTO compliance.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧠 15. Summary – HA/DR for Managed Instance

Feature	Purpose	Automated	Region
Built-in HA	Protects within region	✅	Same
PITR	Recover data at any time	✅	Same
LTR	Archive backups	✅	Same
Failover Groups	DR across regions	✅	Cross-region
Manual Failover	Test/Planned switch	✅	Cross-region

🏁 16. Key Takeaways

- ✅ Always use **Zone Redundant** deployment for mission-critical workloads.
- ✅ **Auto-Failover Groups** = Best practice for DR.
- ✅ Validate **PITR** and **LTR** restores monthly.
- ✅ Run **DR drills quarterly** and document failover time.
- ✅ Monitor replication_lag_sec for DR health.
- ✅ TDE protects all backups automatically.

https://www.sqldbachamps.com

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 7 (Version-2):
High Availability, Backups & Disaster Recovery (HA/DR) in Azure SQL Managed Instance — one of the **most critical modules for SQL Cloud DBAs**.
By the end of this, you'll be able to design **enterprise-grade, zero-data-loss, auto-failover** HA/DR strategies in Azure — *just like an Azure Architect*.

🧱 1. Understanding High Availability (HA) in Managed Instance

Azure SQL Managed Instance is **built on the SQL Server Always On Availability Groups (AG)** architecture — but **fully managed by Microsoft**.
You don't configure AGs or replicas manually — Azure does it for you behind the scenes.

✅ Key Highlights:

Feature	Description
3 replicas per instance	1 Primary + 2 Secondary replicas (synchronous)
Automatic failover	Microsoft detects health & triggers failover
Zero data loss	Synchronous commit between replicas
Automatic patching	Failover during maintenance (invisible to users)
No manual setup	Always On is built-in and hidden from the DBA

⚙️ 2. HA Architecture Diagram (Conceptual)

https://www.sqldbachamps.com

```
        +----------------------------+
        |  Azure SQL Managed Instance |
        +----------------------------+
         |    |      |
+------------+    |      +-------------+
| Primary Replica  |      | Secondary Replica 2 |
| (Readable/Write) |      | (Hidden, sync commit)|
+------------------+      +---------------------+
         |
         +---> Secondary Replica 1 (Sync)
             |
             +--> Auto Failover (0 RPO)
```

✅ Replication is **synchronous** within the same region — ensuring **0 data loss (RPO = 0)** and **fast recovery (RTO < 60s)**.

💡 3. Failover Triggers (Auto-HA)

Azure automatically performs failover when:

Trigger	Cause
Hardware Failure	Node crash or host issue
OS/SQL Patch	Maintenance updates
Software Failure	Process/service crash
Resource Constraint	Memory or I/O stall
Zone Failure	Availability Zone outage (if zone-redundant)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧩 4. Availability Zone Support

- By default, Managed Instance is **zone-redundant** in supported regions.
- This means **replicas are spread across 3 zones**, protecting against data center-level failures.

✅ Configuration:
In portal → while creating MI → "**Zone Redundant Configuration: ON**".

💾 5. Automated Backups

Azure SQL Managed Instance automatically takes **full, differential, and transaction log backups**.

Type	Frequency	Retention	Purpose
Full Backup	Every 7 days	Default: 7–35 days	Baseline restore
Differential	Every 12 hours	Auto	Reduce PITR time
Log Backup	Every 5–10 min	Auto	Point-in-time restore

🧩 5.1 Backup Storage Location

Backups are automatically stored in **RA-GRS (Read-Access Geo-Redundant Storage)** in Azure Blob.
You can't access these files directly — Azure manages them internally.

🧩 5.2 Backup Retention Settings

In the Portal:

- Go to **MI → Backups → Configure Retention**
- You can set **7–35 days** retention per database

✅ For long-term retention (LTR):

- Export backups to **Azure Blob Storage**
- Or use **Azure Backup Vault** for year-long retention.

https://www.sqldbachamps.com

⚙️ 6. Point-In-Time Restore (PITR)

You can restore a database to **any point within retention window** (down to the minute).

🔄 **Restore Steps:**

1. Go to **MI → Databases → Select Database → Restore**
2. Choose "**Point-in-time restore**"
3. Select date & time → Provide new DB name
4. Click **Restore**

Or via PowerShell:

```
Restore-AzSqlInstanceDatabase `
  -FromPointInTimeBackup `
  -TargetInstanceName "sqlmi-prod" `
  -Name "SalesDB" `
  -PointInTime "2025-10-08T08:30:00Z" `
  -ResourceGroupName "RG-SQL-MI"
```

✅ Restore time ≈ size & log chain complexity (usually 5–30 minutes).

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🌍 7. Geo-Replication (DR Across Regions)

Managed Instance supports **Auto-failover groups (FOGs)** for cross-region DR.

Primary Region	Secondary Region	Sync Type	Failover
East US	West US	Asynchronous	Manual / Auto
West Europe	North Europe	Asynchronous	Manual / Auto

🧩 7.1 Failover Group Components

Component	Description
Primary Managed Instance	Main writable instance
Secondary Managed Instance	Read-only replica (DR region)
Listener Endpoint	Single DNS endpoint (auto-switches)
Geo-replication	Async data replication between regions

🧩 7.2 Create Failover Group (Portal)

🔍 Steps:

1. MI → **Failover Groups** → **+ Add Group**
2. Provide:
   - Group Name: mi-prod-fog
   - Partner MI: sqlmi-dr (in another region)
   - Read/Write Listener: auto-generated
   - Failover Policy: *Automatic or Manual*
3. Save ✅

https://www.sqldbachamps.com

Replication starts immediately.

🧩 7.3 T-SQL View for FOG Status

```
SELECT * FROM sys.failover_groups;
SELECT * FROM sys.databases WHERE database_id > 4;
```

🧩 7.4 Test Failover (DR Drill)

To test DR:

```
Switch-AzSqlDatabaseFailoverGroup `
  -ResourceGroupName "RG-MI" `
  -ServerName "sqlmi-prod" `
  -Name "mi-prod-fog"
```

✅ Within ~60 seconds, connections switch to secondary automatically.

🔄 8. Restore Options Summary

Scenario	Restore Option	Tool
Accidental delete	Point-in-time restore	Portal / PowerShell
Region outage	Failover group switch	Portal / PowerShell
Logical corruption	PITR to earlier timestamp	Portal
Audit recovery	Export BACPAC backup	SSMS / Azure Storage

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧮 9. Backup & DR Validation Queries

Check Last Backup Time

```
SELECT
  database_name, backup_finish_date, type_desc
FROM msdb.dbo.backupset
ORDER BY backup_finish_date DESC;
```

Check Replication Health

```
SELECT * FROM sys.dm_geo_replication_link_status;
```

Check Failover Group Status

```
SELECT group_id, partner_server, replication_state_desc
FROM sys.failover_groups;
```

🧠 10. HA/DR Best Practices Checklist

Area	Best Practice
HA	Always choose zone-redundant MI
DR	Use Failover Groups for cross-region protection
Backups	Retention 14–35 days minimum
PITR	Test quarterly
FOG	Run failover drills twice yearly
Security	Encrypt backups (TDE)
Alerts	Set alerts for backup failure / DR lag
RPO/RTO	Validate against business SLA (RPO ≤ 15 min, RTO ≤ 1 hr)

https://www.sqldbachamps.com

🧩 11. Example: Enterprise HA/DR Design

```
+-----------------------------------------------------------+
|  Region 1: East US (Primary)                              |
|  - Managed Instance (MI-Prod)                             |
|  - 3 Replicas (HA)                                        |
|  - Backup to GRS Storage                                  |
+-----------------------------------------------------------+
              | Geo-Replication (Async)
              |
              v
+-----------------------------------------------------------+
|  Region 2: West US (Secondary)                            |
|  - Managed Instance (MI-DR)                               |
|  - Auto-Failover Group                                    |
|  - Read-Only Queries allowed                              |
+-----------------------------------------------------------+
```

✅ Benefits:

- HA (in-region) handled by Azure
- DR (cross-region) with FOG
- PITR + Geo redundancy → Complete 3-layer protection

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🧮 12. Summary – Azure SQL Managed Instance HA/DR in a Nutshell

Feature	Description	Default
HA	3 replicas, synchronous commit	✅
Zone redundancy	Cross-availability-zone protection	✅ (supported regions)
Backups	Full, diff, log	✅
PITR	Restore to any point in retention window	✅
Failover Groups	Cross-region replication	Optional
Auto-failover	During maintenance / outage	✅
Manual failover	For DR drills	✅
Backup retention	7–35 days	Configurable

✅ You now fully understand **HA, backups, PITR, and geo-replication** for Managed Instance — a *core Azure SQL DBA skill.*

https://www.sqldbachamps.com

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Module 8: Azure SQL Managed Instance — Cost Optimization, Scaling, and Performance Tier Management.

This module is all about **getting the best performance for your money** — critical for production workloads, budgets, and cloud cost governance.

🧱 1. Azure SQL Managed Instance Pricing Models

Azure SQL MI is priced primarily by **Compute + Storage**. There are two models:

Model	Description	Use Case
vCore-based (Recommended)	Pay for virtual cores + storage separately	Predictable performance, enterprise workloads
DTU-based (Deprecated for MI)	Bundled DTU metric	Legacy apps; mostly for Azure SQL Database

⚡ Compute Tiers (vCore Model)

Tier	Max vCores	IO Latency	Use Case
General Purpose (GP)	8–80	Moderate	Most OLTP workloads, budget-friendly
Business Critical (BC)	4–128	Low (<1ms)	High IOPS, low latency, mission-critical
Hyperscale	Not supported for MI	N/A	Only Azure SQL Database

✅ BC includes **local SSD storage + zone redundancy**.
✅ GP uses **remote Azure Premium Storage** — slightly higher IO latency.

https://www.sqldbachamps.com

🧮 2. Storage Sizing and Cost Optimization

- **Managed storage** grows automatically but you **pay per GB**.
- Types of storage in MI:

Storage	Tier	Notes
Data files	GP: Premium storage	IOPS depends on vCores
Log files	Premium storage	Ensure low latency
TempDB	Managed, automatically sized	Shared across DBs in instance

Optimization Tips:

1. **Remove unused tables or old data** → reduces storage cost.
2. **Use Partitioning** → reduces IO for queries, improves cost efficiency.
3. **Set retention policies** → archive old backups, avoid unnecessary LTR.

⚡ 3. Scaling vCores (Vertical Scaling)

Managed Instance allows **dynamic scaling** of compute resources:

Steps (Portal):

1. MI → **Compute + Storage**
2. Change **vCores** or **Tier**
3. Apply → Instance restarts (~5–10 mins)

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

PowerShell Example:

```
Set-AzSqlInstance `
 -ResourceGroupName "RG-SQLMI" `
 -Name "sqlmi-prod" `
 -SkuName "GP_Gen5_16" `
 -StorageSizeInGB 1024
```

✅ Tip: Scale **during low usage hours** to reduce downtime impact.

🔄 4. Horizontal Scaling (Read Scale-Out)

- **Business Critical** tier supports **read-only replicas** for reporting workloads.
- App can connect to secondary endpoint for **read-heavy queries**, reducing primary load.

Benefits:

- No impact on primary for heavy reporting.
- Saves vCore usage on primary, indirectly saving cost.

💰 5. Cost-Saving Strategies

5.1 Reserved Capacity

- Commit 1–3 years → up to **33% savings** vs pay-as-you-go.
- Works for **vCore-based MI**.

5.2 Auto-Pause (for dev/test)

- Currently, **Auto-Pause** exists only for **Azure SQL Database**, not MI.
- But **dev/test MI** can use **smaller vCores + GP tier** for cost saving.

https://www.sqldbachamps.com

5.3 Tier Optimization

- **General Purpose** for OLTP with moderate workload.
- **Business Critical** only for high IOPS / SLA-critical workloads.
- Switch tiers dynamically as workload changes.

5.4 Storage Tiering

- Avoid over-provisioning storage.
- Start small, grow as needed (auto-grow enabled).

🧮 6. Example Cost Comparison

Scenario	Tier	vCores	Storage	Est. Cost/month
Dev/Test	GP	4	512 GB	~$1,200
Production	GP	16	1 TB	~$5,500
Production	BC	16	1 TB	~$9,000

💡 Always **estimate using Azure Pricing Calculator** for accurate monthly cost.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

🛠 7. Performance Tier Management Tips

Tip	Reason
Start with **GP tier**, then upgrade to BC	Cost efficiency
Use **Query Store** to analyze workload	Helps right-size vCores
Monitor **CPU%, IO latency**	Avoid over-provisioning
Implement **Read Scale-Out**	Offload reporting, save primary vCores
Automate scaling for predictable peaks	Reduce manual cost adjustments

📊 8. Monitoring Usage vs Cost

- Azure Portal → **Metrics & Cost Analysis**
- Monitor:
  - **vCore utilization** (CPU%)
  - **IO consumption**
  - **Storage growth**
- Set alerts when **storage > 80% of provisioned** to avoid over-provisioning.

🧠 9. Real-Time DBA Best Practices for Cost Optimization

1. **Right-tier selection:** GP for normal OLTP, BC for low-latency, high-IOPS.
2. **Auto-scale vCores for peak load only**.
3. **Offload reporting to read-only replicas**.
4. **Archive historical data & backups** → reduce storage charges.
5. **Use Reserved Instances** for 1–3 year predictable workloads.
6. **Monitor and tune queries** → reduces unnecessary compute.
7. **Audit and remove unused DBs / dev/test environments**.

https://www.sqldbachamps.com

🔑 10. Summary – Cost Optimization & Scaling

Area	Strategy
Tier selection	GP vs BC based on SLA & IO
vCores scaling	Vertical scaling (dynamic)
Read scale-out	Offload reporting to secondary
Storage	Start small, auto-grow, archive old data
Reserved capacity	1–3 years → cost savings
Monitoring	Track CPU, IO, storage growth

✅ Key takeaway: **Right-size, monitor, scale dynamically, archive unused resources** = optimum cost vs performance.

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

Ultimate Azure SQL Managed Instance DBA Playbook / Cheat Sheet — consolidating **all 8 modules** into one single, production-ready reference.

This include:

- Architecture diagrams & high-level overview
- Security & authentication commands
- Backup, HA/DR scripts
- Monitoring & performance tuning commands
- Cost optimization tips & tier management
- Quick commands for daily DBA checks

Here's a **structured cheat sheet**:
Azure SQL Managed Instance DBA Ultimate Playbook / Cheat Sheet

1 Architecture Overview

- **Deployment:** Always inside a **VNet** → optionally with public endpoint
- **High Availability:** Built-in **Always On Availability Group** (Primary + Secondary replicas)
- **Tiers:** General Purpose (GP), Business Critical (BC)
- **Storage:** Managed, auto-grow; Premium SSD (BC) or remote storage (GP)
- **Security Layers:**
    1. Network (VNet, NSG, Private Endpoint)
    2. Authentication (SQL, AAD, Managed Identity)
    3. Encryption (TDE, Always Encrypted, TLS)
    4. Auditing & Threat Detection (Defender for SQL)

https://www.sqldbachamps.com

2 Security & Authentication

SQL Login

```
CREATE LOGIN [appuser] WITH PASSWORD = 'Str0ngP@ss!';
CREATE USER [appuser] FOR LOGIN [appuser];
ALTER ROLE db_datareader ADD MEMBER [appuser];
```

Azure AD Login

```
CREATE USER [praveen@domain.com] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [praveen@domain.com];
```

Managed Identity (Access Azure Resource)

```
CREATE DATABASE SCOPED CREDENTIAL [AzureBlobCred]
WITH IDENTITY = 'Managed Identity';
```

Transparent Data Encryption (TDE)

```
SELECT db_name(database_id) AS DBName, encryption_state, key_algorithm, key_length
FROM sys.dm_database_encryption_keys;
```

Always Encrypted (Column-Level)

```
CREATE COLUMN MASTER KEY [CMK_AzureKeyVault]
WITH KEY_STORE_PROVIDER_NAME = N'AZURE_KEY_VAULT',
KEY_PATH = N'https://yourvault.vault.azure.net/keys/CustomerKey/';
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator


```
CREATE COLUMN ENCRYPTION KEY [CEK_1]
WITH VALUES (COLUMN_MASTER_KEY = [CMK_AzureKeyVault], ALGORITHM = 'RSA_OAEP');

CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  SSN NVARCHAR(11) COLLATE Latin1_General_BIN2 ENCRYPTED WITH (
    COLUMN_ENCRYPTION_KEY = [CEK_1],
    ENCRYPTION_TYPE = DETERMINISTIC,
    ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256') NOT NULL,
  Name NVARCHAR(50)
);
```

3 Auditing & Threat Protection

Enable Auditing

Portal → SQL MI → Auditing → Enable → Destination: Log Analytics / Storage

Kusto Query: Top Audit Events

```
AzureDiagnostics
| where Category == "SQLSecurityAuditEvents"
| where LogicalServerName_s == "sqlmi-demo2025"
| project TimeGenerated, statement_s, succeeded_s
```

Enable Defender for SQL

Portal → MI → Microsoft Defender → Enable

4 Monitoring & Performance

https://www.sqldbachamps.com

Metrics

- CPU %, IO latency, Storage space, Sessions, vCore utilization

Alerts

- Azure Alerts → CPU > 80%, Storage > 80%, Long-running queries

Query Store

```
ALTER DATABASE [SalesDB] SET QUERY_STORE = ON;
ALTER DATABASE [SalesDB]
SET QUERY_STORE (OPERATION_MODE = READ_WRITE, CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30));
```

Top CPU Queries

```
SELECT TOP 10
   qs.query_id, qt.query_sql_text, rs.avg_cpu_time, rs.count_executions
FROM sys.query_store_query_text qt
JOIN sys.query_store_query qs ON qt.query_text_id = qs.query_text_id
JOIN sys.query_store_runtime_stats rs ON qs.query_id = rs.query_id
ORDER BY rs.avg_cpu_time DESC;
```

Extended Events: Capture Deadlocks

```
CREATE EVENT SESSION [Deadlocks]
ON SERVER
ADD EVENT sqlserver.lock_deadlock
ADD TARGET package0.event_file (SET filename = 'deadlocks.xel');
ALTER EVENT SESSION [Deadlocks] ON SERVER STATE = START;
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

DMVs Quick Check

```
-- CPU-heavy queries
SELECT TOP 10 total_worker_time/1000 AS CPU_ms, execution_count, total_worker_time/execution_count AS Avg_CPU,
SUBSTRING(qt.text,1,200) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY CPU_ms DESC;
```

5 HA / DR & Backups

Point-in-Time Restore

```
RESTORE DATABASE [SalesDB_Restore_2025]
FROM DATABASE_SNAPSHOT = 'SalesDB'
AS OF '2025-10-09T08:30:00';
```

Long-Term Retention (LTR) Policy

```
Set-AzSqlDatabaseBackupLongTermRetentionPolicy `
  -ResourceGroupName "RG-SQL" `
  -ServerName "sqlmi-prod" `
  -DatabaseName "SalesDB" `
  -WeeklyRetention "P12W" `
  -MonthlyRetention "P12M" `
  -YearlyRetention "P10Y"
```

https://www.sqldbachamps.com

Auto-Failover Group

```
New-AzSqlDatabaseFailoverGroup `
  -ResourceGroupName "RG-Prod" `
  -ServerName "sqlmi-eastus" `
  -PartnerServerName "sqlmi-westus" `
  -FailoverGroupName "FG-MI-PROD" `
  -FailoverPolicy Automatic `
  -GracePeriodWithDataLossHours 1 `
  -AddDatabase "SalesDB","FinanceDB"
```

Manual Failover

```
Switch-AzSqlDatabaseFailoverGroup `
  -ResourceGroupName "RG-Prod" `
  -ServerName "sqlmi-eastus" `
  -FailoverGroupName "FG-MI-PROD"
```

Backup Validation

```
DBCC CHECKDB ('SalesDB_Restore') WITH NO_INFOMSGS;
```

Praveen Madupu
Mb: +91 98661 30093
Sr. Database Administrator

6 Cost Optimization & Scaling

vCore Scaling

```
Set-AzSqlInstance `
 -ResourceGroupName "RG-SQLMI" `
 -Name "sqlmi-prod" `
 -SkuName "GP_Gen5_16" `
 -StorageSizeInGB 1024
```

Tips:

- **Start with GP tier**, upgrade to BC if needed
- **Read Scale-Out** → offload reporting
- **Archive historical DBs** → reduce storage
- **Reserved Capacity (1–3 yrs)** → save up to 33%
- **Monitor vCore, CPU, IO** → avoid over-provisioning

7 Daily / Weekly DBA Checklist

Frequency	Task
Daily	CPU%, IO latency, active sessions, blocking
Daily	Query Store for regressed queries
Daily	Extended Events / DMVs for deadlocks
Weekly	Validate backups, PITR restores
Weekly	Audit logs review
Weekly	Threat detection alerts review
Monthly	DR drill via Auto-Failover Group
Monthly	Storage growth & optimization review
Quarterly	Tier / vCore optimization & reserved capacity review

https://www.sqldbachamps.com

8 Key Best Practices

- Use **Azure AD authentication** over SQL logins
- Enable **TDE + Always Encrypted** for sensitive data
- Offload reporting to **read-only replicas**
- Implement **Query Store + Automatic Tuning**
- Monitor **metrics + alerts** continuously
- Use **Zone Redundant + Failover Groups** for mission-critical workloads
- Optimize **compute + storage** dynamically to save cost

✅ This playbook now gives you **access to commands, portal steps, architecture, monitoring, HA/DR, and cost optimization** — everything a real-time SQL Managed Instance DBA needs.